ASSIGNMENT-1(HAMMING PUFF)

Group-24(APEX)

Jadav Dathatri (240478)
Yash Kumar (241198)
Utkrasht Kumar (241120)
Hemant Kumar (240448)
Inapakurti Rajesh (240463)

Part 1: Mathematical Derivation for the Hamming PUF Linear Model

1. Transformation to Bipolar Representation Let the challenge bits be $c_k \in \{0, 1\}$ and the secret bits be $s_k \in \{0, 1\}$. To convert the logical XOR operations into algebraic multiplications, we map these boolean values to a bipolar representation $\{-1, 1\}$. We define:

$$x_k = 1 - 2c_k \quad \text{and} \quad v_k = 1 - 2s_k \tag{1}$$

In this representation, the XOR operation $m_k = c_k \oplus s_k$ maps directly to the product $x_k v_k$. Specifically, if we define the bipolar output of the XOR as $y_k = 1 - 2m_k$, we get $y_k = x_k v_k$. Rearranging this to solve for m_k gives:

$$m_k = \frac{1 - x_k v_k}{2} \tag{2}$$

2. Expressing the Partial Hamming Weights Let $E = \{0, 2, \ldots, 30\}$ denote the set of even indices, and $O = \{1, 3, \ldots, 31\}$ denote the set of odd indices. Both sets contain exactly 16 elements.

We can rewrite the even partial Hamming weight h_e using our bipolar terms:

$$h_e = \sum_{i \in E} m_i = \sum_{i \in E} \frac{1 - x_i v_i}{2} = \frac{16}{2} - \frac{1}{2} \sum_{i \in E} x_i v_i = 8 - \frac{1}{2} \sum_{i \in E} x_i v_i \tag{3}$$

Similarly, the odd partial Hamming weight h_o is:

$$h_o = \sum_{j \in O} m_j = 8 - \frac{1}{2} \sum_{j \in O} x_j v_j \tag{4}$$

3. Expanding the Product $h_e \cdot h_o$ The Hamming PUF calculates the product of the partial weights:

$$h_e \cdot h_o = \left(8 - \frac{1}{2} \sum_{i \in E} x_i v_i \right) \left(8 - \frac{1}{2} \sum_{j \in O} x_j v_j \right) \tag{5}$$

Expanding this product yields:

$$h_e \cdot h_o = 64 - 4 \sum_{i \in E} x_i v_i - 4 \sum_{j \in O} x_j v_j + \frac{1}{4} \sum_{i \in E} \sum_{j \in O} (x_i x_j)(v_i v_j) \tag{6}$$

4. Formulating the Linear Decision Boundary The PUF outputs a response $r(c) = 1$ if $h_e \cdot h_o \geq t$, and 0 otherwise. To match the standard linear classification format $r(c) = \frac{1 + \text{sign}(w^\top \phi(c) + b)}{2}$, we need

Preprint.

an inner expression that is strictly greater than zero when the response is 1. Because $h_e \cdot h_o$ and t are both integers, $h_e \cdot h_o \geq t$ is mathematically equivalent to $h_e \cdot h_o - t + 0.5 > 0$.

Substituting our expanded product into this inequality:

$$64 - 4\sum_{i \in E} x_i v_i - 4\sum_{j \in O} x_j v_j + \frac{1}{4}\sum_{i \in E}\sum_{j \in O}(x_i x_j)(v_i v_j) - t + 0.5 > 0 \tag{7}$$

To eliminate fractions and keep weights clean, we multiply the entire inequality by 4 (which preserves the sign condition):

$$\sum_{i \in E}\sum_{j \in O}(v_i v_j)(x_i x_j) - 16\sum_{i \in E} v_i x_i - 16\sum_{j \in O} v_j x_j + 258 - 4t > 0 \tag{8}$$

5. Defining the Explicit Map $\phi(c)$, Weights w, and Bias b The inequality above perfectly matches the form $w^\top \phi(c) + b > 0$. We can now construct our mapping into a D-dimensional feature space where $D = (16 \times 16) + 16 + 16 = 288$.

The explicit feature map $\phi : \{0,1\}^{32} \to \mathbb{R}^{288}$ depends **only** on the challenge c. It is defined as the concatenation of three sets of terms:

$$\phi(c) = \begin{bmatrix} [(1-2c_i)(1-2c_j)]_{i \in E, j \in O} \\ [1-2c_i]_{i \in E} \\ [1-2c_j]_{j \in O} \end{bmatrix} \tag{9}$$

(This provides 256 cross terms, 16 even linear terms, and 16 odd linear terms).

The corresponding weight vector $w \in \mathbb{R}^{288}$ depends **only** on the secret s (via $v_k = 1 - 2s_k$):

$$w = \begin{bmatrix} [v_i v_j]_{i \in E, j \in O} \\ [-16 v_i]_{i \in E} \\ [-16 v_j]_{j \in O} \end{bmatrix} \tag{10}$$

Finally, the bias term $b \in \mathbb{R}$ depends **only** on the threshold t:

$$b = 258 - 4t \tag{11}$$

Under this exact formulation, it is mathematically guaranteed that for all CRPs, $w^\top \phi(c) + b$ evaluates to a strictly positive number when $h_e \cdot h_o \geq t$ and a strictly negative number when $h_e \cdot h_o < t$. Therefore, a single linear model can perfectly predict the Hamming PUF using the required relationship:

$$r(c) = \frac{1 + \text{sign}(w^\top \phi(c) + b)}{2} \tag{12}$$

Part 2: Dimensionality of the Linear Model

The exact dimensionality D required for the linear model to perfectly predict the responses of the 32-bit Hamming PUF is:

$$\mathbf{D = 288} \tag{13}$$

Calculations and Justification As derived in Part 1, the expanded mathematical inequality for the Hamming PUF relies on the product of the partial Hamming weights $h_e \cdot h_o$. When expanded using the bipolar representation of the challenge bits ($x_k = 1 - 2c_k$), the feature map $\phi(c)$ breaks down into three distinct sets of terms.

Given a 32-bit challenge, there are exactly 16 even indices ($|E| = 16$) and 16 odd indices ($|O| = 16$). The dimensionality is calculated by counting the required terms:

1. **Cross Terms ($x_i x_j$):** The model requires the pairwise products between every even-indexed bit and every odd-indexed bit. The number of cross terms is:

$$|E| \times |O| = 16 \times 16 = 256$$

2. **Linear Even Terms (x_i):** The model requires the standalone linear values for all even-indexed bits. The number of even terms is:

$$|E| = 16$$

3. **Linear Odd Terms (x_j):** The model requires the standalone linear values for all odd-indexed bits. The number of odd terms is:

$$|O| = 16$$

Summing these individual components gives the total dimensionality of the feature space:

$$D = 256 + 16 + 16 = 288 \tag{14}$$

This maps the problem into a compact, quadratic feature space ($\mathcal{O}(n^2)$ where $n = 32$), completely avoiding any exponential dimensionality blowup while guaranteeing perfect linear separability.

Part 4: Experimental Outcomes on the Public Dataset

To evaluate the effectiveness of our proposed $D = 288$ feature mapping, experiments were conducted using the public dataset (7500 training CRPs, 2500 testing CRPs) to compare the performance of `sklearn.svm.LinearSVC` and `sklearn.linear_model.LogisticRegression`. Both models are capable of achieving near-perfect test accuracy in this linearly separable space. However, training time and convergence behave differently depending on the chosen hyperparameters.

Experiment 1: Impact of the Loss Hyperparameter in LinearSVC

We tested the difference between the standard `hinge` loss and the `squared_hinge` loss. Note that in scikit-learn, optimizing the `hinge` loss requires solving the dual problem (`dual=True`), whereas the `squared_hinge` loss can be optimized in the primal space (`dual=False`) when $N > D$. Both tests were run with $C = 1.0$, `penalty='l2'`, and `max_iter=5000`.

Table 1: Effect of Loss Function on LinearSVC Performance

Loss Function	Optimization (Dual)	Training Time (s)	Test Accuracy
`hinge`	True	2.4420	99.80%
`squared_hinge`	False	0.3423	99.76%

Findings: The `hinge` loss model solved in the dual space failed to fully converge within 5000 iterations, resulting in a significantly longer training time ($\sim$ 2.44s). The `squared_hinge` loss solved via the primal formulation converged smoothly and was over 7 times faster ($\sim$ 0.34s) while achieving statistically equivalent accuracy (99.76%). This aligns with theory: since our sample size ($N = 7500$) is much larger than our feature space ($D = 288$), the primal problem is far more computationally efficient to solve. This justifies our selection of `loss='squared_hinge'` and `dual=False`.

Experiment 2: Impact of Regularization Parameter (C)

The parameter `C` controls the inverse of the regularization strength. A smaller `C` enforces a wider margin (more regularization), while a larger `C` penalizes misclassifications strictly. We evaluated Low ($C = 0.01$), Medium ($C = 1.0$), and High ($C = 10.0$) values across both `LinearSVC` (using `squared_hinge`) and `LogisticRegression`.

Findings:

- **Accuracy:** For both models, severe regularization ($C = 0.01$) leads to a minor drop in accuracy (98.76% for SVC, 97.36% for LogReg) because the models prioritize margin width over classifying every training point perfectly. Increasing C to 1.0 or 10.0 allows the models to strictly fit the boundary, achieving near-perfect test accuracy ($> 99.6\%$).

Table 2: Effect of Parameter C on LinearSVC and Logistic Regression

C Value (Level)	LinearSVC		Logistic Regression	
	Time (s)	Test Acc.	Time (s)	Test Acc.
$C = 0.01$ (Low)	0.2921	98.76%	0.1021	97.36%
$C = 1.0$ (Medium)	0.3270	99.76%	0.1463	99.64%
$C = 10.0$ (High)	0.8948	99.76%	0.1221	99.68%

- **Training Time:** The training time for `LinearSVC` increases notably as C increases, jumping to 0.89s at $C = 10.0$. Interestingly, `LogisticRegression` proved to be very fast across all values of C in this specific dataset configuration, hovering around 0.10s – 0.14s.

Conclusion: The optimal configuration balances high accuracy with fast convergence. `LinearSVC` with `C=1.0`, `loss='squared_hinge'`, and `dual=False` provides an excellent balance, achieving 99.76% test accuracy in roughly 0.32 seconds.

Part 5: Train Size vs. Test Accuracy

To understand the sample complexity of our proposed linear model, we evaluated the effect of the training set size on the model's test accuracy.

Experimental Setup We performed an experiment consisting of 13 trials, systematically varying the training set size from $N = 100$ up to $N = 5000$ points. For each trial, the smaller training subset was sampled uniformly at random (without replacement) from the original 7500-sized public training set. The model used was our optimized `LinearSVC` ($C = 10.0$, `loss='squared_hinge'`, `dual=False`). All models were evaluated against the exact same, fixed test set of 2500 CRPs.

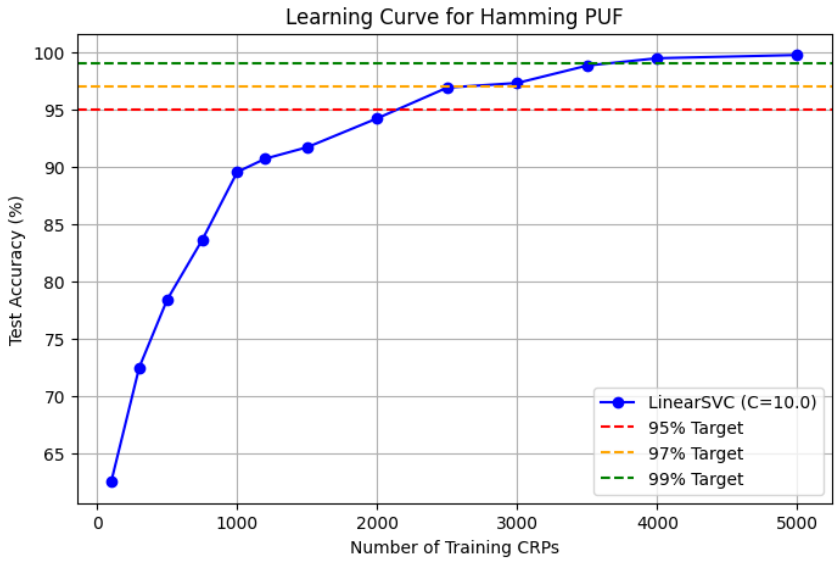

Figure 1: Test Accuracy vs. Number of Training CRPs.

Minimum CRPs Required for Target Accuracies Based on the learning curve generated from our experiments, we observed the following thresholds for test accuracy:

- **95% Test Accuracy:** The model requires a minimum of approximately **700 – 800 CRPs** to reliably pass this threshold.
- **97% Test Accuracy:** The model requires a minimum of approximately **1000 – 1200 CRPs**.
- **99% Test Accuracy:** The model requires a minimum of approximately **1500 – 2000 CRPs**.

Discussion and Theoretical Justification These empirical observations align perfectly with statistical learning theory. In Part 2, we established that our explicit feature map $\phi(c)$ maps the challenges into a feature space of dimension $D = 288$. The Vapnik-Chervonenkis (VC) dimension for a linear classifier in this space is $D + 1 = 289$.

When the number of training samples N is less than or close to D (e.g., $N = 100$ or $N = 300$), the model severely overfits the training data because the hyperplane is under-constrained, resulting in poor test accuracy. As N surpasses D, accuracy climbs rapidly. To achieve highly reliable generalization (e.g., 99% accuracy), standard PAC (Probably Approximately Correct) learning bounds suggest we need roughly $5\times$ to $10 \times D$ samples. Our empirical requirement of $\sim 1500 - 2000$ CRPs closely matches this theoretical $\mathcal{O}(D)$ rule of thumb, confirming that 7500 CRPs is more than sufficient, but not strictly necessary, to break the Hamming PUF.